

Algoritmos y Estructuras de Datos

Pilas y Colas

Pilas

Una pila es una estructura de tipo LIFO (Last In, First Out o último que entra, primero que sale) porque puedo agregar elementos en un extremo y sacar elementos de ese mismo extremo (en el tope de la pila). Las restricciones básicas que tiene son que no puedo agregar y sacar elementos de cualquier lado, sino que debe ser el de arriba.

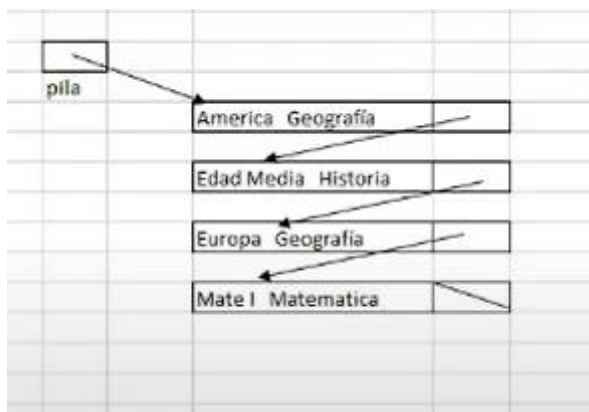
Por lo general voy a usar una pila como estructura auxiliar cuando quiero guardar datos y después los quiero recuperar en el orden inverso de como los ingresé.

Vamos a hacer una pila de libros utilizando el struct Libro, donde ponemos título y materia. A cada libro debemos agregarle un elemento más que apunte al siguiente. Esto no lo vamos a hacer dentro del struct, sino a través de un nodo que va a tener un miembro llamado info (de tipo Libro) o datos y un puntero sig que apunta al siguiente nodo de la estructura.

```
struct Libro
{
    string titulo;
    string materia;
};

struct NodoPila
{
    Libro info;
    NodoPila*sig;
};
```

Entonces las estructuras dinámicas van a estar compuestas por nodos, que van a tener la información de qué va a ser la pila y el puntero que me enlace ese nodo con otro nodo de la estructura. Asimismo, debo tener un puntero externo que va a apuntar al tope de la pila, el nodo del tope de la pila va a apuntar al que esté abajo, y así sucesivamente. El que esté debajo de todo va a tener NULL.



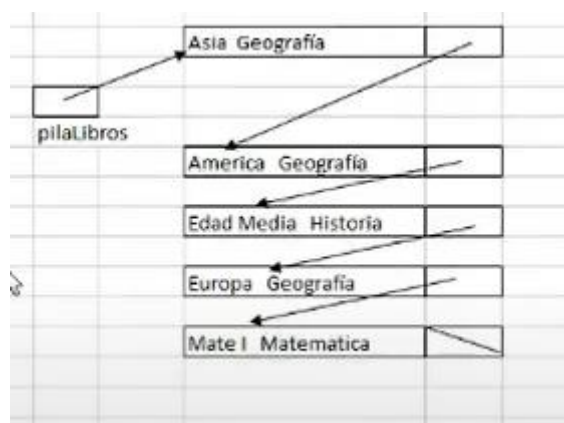
Para apilar vamos a usar la función apilar. Esta función recibe la pila y que es lo que quiere apilar, en este caso un libro. Entonces recibe el puntero a la pila que la debo pasar por referencia y una variable lib de tipo Libro.

En la función vamos a declarar una variable puntero a nodo pila *p, a la cual le asigno new NodoPila. Luego, en p->info asigno lib, o sea, lo que tiene lib lo guardo en la parte info del nodo de pila. Para enlazarlo con el otro nodo debo asignarle a p->sig el contenido de pila, para que apunte a lo mismo a lo que está apuntando pila, que es el nodo que está arriba de todo. Luego, a la variable pila le asignamos p para que apunte a al tope de la pila.

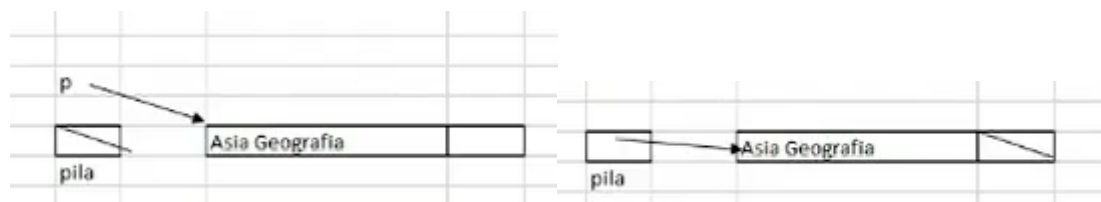
```
int main()
{
    NodoPila *pilaLibros;
    Libro libro;
    //...

    cin>>libro.titulo;
    cin>>libro.materia;
    apilar(pilaLibros, libro);
}

void apilar(NodoPila*&pila, Libro lib)
{
    NodoPila*p=new NodoPila;
    p->info=lib;
    p->sig=pila;
    pila=p;
}
```



También puede pasar que apilemos a una pila vacía. Pila en este caso va a tener NULL.



En el main, cuando recién se arranca a apilar es muy importante que la pila esté inicializada en NULL, porque sino va a tener basura y no puedo apilar.

```
int main()
{
    NodoPila *pilaLibros=NULL;
    Libro libro;
    cin>>libro.titulo;
    while(libro.titulo!="fin")
    {
        cin>>libro.materia;
        apilar(pilaLibros, libro);
        cin>>libro.titulo;
    }
}
```

Ahora vamos a hacer una función desapilar. Por lo general, cuando queremos desapilar es para hacer algo sobre los datos de la pila. La idea principal en el main es que vamos a desapilar y mostrar.

La función desapilar va a necesitar la referencia a pila y devuelve un libro, que es el que está en el tope de la pila. Dentro de la función voy a usar una variable Libro lib para guardar el libro que me devuelve. Además, debo sacar el nodo. Guardo en una variable p puntero auxiliar de tipo NodoPila lo que está en pila, es decir p=pila (p apunta a pila). Luego asigno a lib la info a la que apunta p (lib=p->info). Ahora a pila tiene que apuntar al sig que está apuntado por p (pila=p->sig). Ahora si puedo eliminar lo que está apuntado por p para sacarlo de la memoria.

Por último hacemos un return lib.

```
Libro desapilar(NodoPila*&pila)
{
    Libro lib;
    NodoPila*p;
    //lib=pila->info;
    p=pila;
    lib=p->info;
    pila=p->sig;
    delete p;
    return lib;
}
```

No puedo desapilar una pila vacía, puesto que pila es NULL. Esto lo reflejo en el while del main.

```

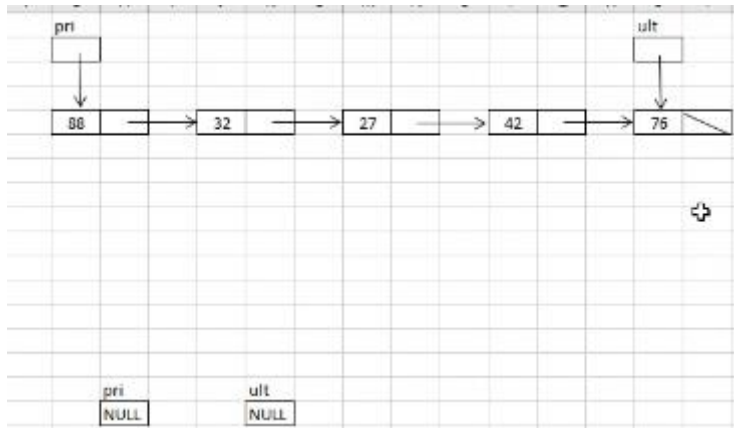
int main()
{
    NodoPila *pilaLibros=NULL;
    Libro libro;
    cin>>libro.titulo;
    while(libro.titulo!="fin")
    {
        cin>>libro.materia;
        apilar(pilaLibros, libro);
        cin>>libro.titulo;
    }
    while(pilaLibros!=NULL)
    {
        libro=desapilar(pilaLibros);
        cout<<libro.titulo<<" - "<<libro.materia<<endl;
    }
}

```

Acá lo que hacemos es guardar en libro el libro que se desapiló y después lo mostramos, siempre y cuando la pila sea distinta de NULL.

Colas

En una cola tenemos elementos enlazados y cada elemento tiene un puntero a otro. Tenemos dos punteros externos, uno al primer elemento y otro al último. Tiene restricciones también. Los elementos se ponen en un extremo y se sacan en otro. Es una estructura FIFO (First In, First Out). El último de la cola en el puntero tiene NULL.



Hay un `NodoCola` con un `int` llamado `info` y un `NodoCola` puntero al siguiente. Si yo tengo la cola vacía los dos punteros principales están en NULL.

Las dos funciones principales son `encolar` y `desencolar`. El `encolar` agrega elementos en el último y `desencolar` devuelve el primero.

La función `encolar` recibe el puntero al primero y al último de la cola pasados por referencias porque uno de los dos se van a modificar seguro (el primero en caso de que la cola esté vacía, sino va a seguir igual). Supongamos que recibimos el entero 33. Para agregar un elemento a la cola necesitamos un nodo de memoria. Entonces declaramos una variable `NodoCola *p` y hacemos `p = new NodoCola`. De esta forma pedimos memoria. Luego completamos los datos haciendo `p->info = nro` y a `p->sig = NULL` porque va a ser el último elemento de la cola y este siempre tiene NULL. Ahora tenemos que decirle al que antes era el último que ahora tiene otro atrás. Modificamos el `sig` del último haciendo `ult->sig = p`, así ahora apunta a lo mismo que tiene `p`. Después hacemos `ult = p` así apunta al último de la cola.

```
void encolar (NodoCola*&pri, NodoCola*&ult, int nro)
{
    NodoCola*p;
    p = new NodoCola;
    p->info = nro;
    p->sig = NULL;
    ult->sig = p;
    ult = p;
}
```

Si encolamos en una cola vacía, `pri` y `ult` están en NULL. En este caso no podemos hacer el `ult->sig = p` porque está en NULL. Debemos poner un `if` para saber si la cola está vacía o no.

```

void encolar(NodoCola*&pri, NodoCola*&ult, int nro)
{
    NodoCola*p;
    p=new NodoCola;
    p->info=nro;
    p->sig=NULL;
    if(ult!=NULL)
        ult->sig=p;
    else
        pri=p;
    ult=p;
}

```

Si ult es NULL tengo que modificar pri=p, así pri deja de tener NULL y apunta a p.

Desencolar

La función desencolar va a devolver el elemento que está sacando de la cola con un return. Va a sacar siempre el primero.

Definimos un int dato que vamos a devolver, asimismo usamos un puntero auxiliar que apunte al nodo que vamos a eliminar (NodoCola *p) y hacemos p=pri. Luego hacemos dato=p->info o pri->info. Pri tiene que apuntar al segundo ahora, entonces pri=p->sig o pri->sig. Ahora que el primero ya está afuera de la cola hacemos un delete p para que no ocupe espacio en memoria innecesario. Elimino el espacio de memoria apuntado por p. Cuando termina la función desaparece p. Pri está pasado por referencia porque cambió.

```

int desencolar(NodoCola*&pri, NodoCola*&ult)
{
    int dato;
    NodoCola *p;
    p=pri;
    dato=p->info; //pri->info;
    pri=p->sig; //pri->sig;
    delete p;
    return dato;
}

```

Si en la cola tenemos un solo elemento, entonces la misma tiene que quedar vacía. En este caso ult va a quedar con basura porque apunta a una dirección de memoria que eliminé, entonces debemos ponerle NULL. Para ello hago un if averiguando si pri quedó en NULL, entonces a ult también le voy a tener que poner NULL. Así, va a quedar la cola vacía.

```

int desencolar(NodoCola*&pri, NodoCola*&ult)
{
    int dato;
    NodoCola *p;
    p=pri;
    dato=p->info; //pri->info;
    pri=p->sig; //pri->sig;
    delete p;
    if(pri==NULL)
        ult=NULL;
    return dato;
}

```

No puedo desencolar de una cola vacía. Cuando pri o ult está en NULL quiere decir que la cola ya está vacía.